

Metodo A

Triage Funnel

Specifica tecnica punto per punto: cosa fa il sistema oggi, cosa farà con Metodo A, e perché ogni cambiamento produce un beneficio misurabile.

*Numeri ancorati al codice in produzione e ai log della pratica del 29 aprile (ZIP: 652 file, 381 MB).
Documento predisposto per la presentazione al cliente.*

INDICE DEI CONTENUTI

§1	Lettura dei PDF digitali	2
§2	Classificazione dei documenti	3
§3	Invio del prompt universale	4
§4	Upload delle immagini per OCR	5
§5	Generazione dell'output (Streaming)	6
§6	Costruzione del Word	7
§7	Gestione errori e crash silenziosi	8
§8	Telemetria real-time per l'utente	9
§9	Sintesi numerica — Prima / Ora	10
§10	Riepilogo per il cliente	11

Data 30 aprile 2026

Riferimento pratica ZIP 29 aprile — 652 file, 381 MB

Stato Documento operativo per il cliente

CONFIDENZIALE

SEZIONE 01

Letture dei PDF digitali

▲ PRIMA

Quando si carica una ZIP, ogni PDF al suo interno viene aperto con la libreria `PyPDF2` per provare a estrarre il testo. Se il testo estratto è inferiore a 15 caratteri, il PDF viene marcato come "scansionato" e mandato al motore OCR di Gemini Vision.

Il problema: PyPDF2 ha estrazione testo poco affidabile su PDF italiani con caratteri speciali, accenti, firme digitali (P7M) e su file con doppio layer (testo + immagine sovrapposta). In molti casi restituisce stringhe corrotte o vuote, e il sistema li manda erroneamente in OCR. Sulla ZIP del 29 aprile, su 318 PDF marcati per OCR, circa 200–250 erano in realtà già digitali: il testo era estraibile con una libreria migliore.

◆ ORA CON METODO A

Sostituiamo `PyPDF2` con `pypdfium2`, la libreria di estrazione testo di Google Chrome. È circa **30 volte più veloce** e ha riconoscimento corretto di font, encoding e firme digitali. Estrae il testo nativo di un PDF in **0,004 secondi** contro i 3–5 secondi di OCR Vision.

Il sistema controlla:

- Se `pypdfium2` estrae più di una soglia di caratteri (es. 200), il PDF è digitale → testo nativo, nessun OCR.
- Se estrae meno → conferma che il PDF è scansionato, va in OCR Vision.

→ PERCHÉ MIGLIORA

- **Riduzione drastica delle chiamate OCR:** da circa 318 a circa 60–80 sulla ZIP boss. Le chiamate evitate sono API Gemini risparmiate, ognuna costa token.
- **Qualità dei testi superiore:** l'OCR di immagini introduce errori di lettura su tabelle, numeri, sigle. Il testo nativo è perfetto.
- **Velocità:** ~250 file × 4 secondi (OCR) sostituiti da ~250 file × 0,01 secondi (nativo) = circa **16 minuti tagliati** dalla pipeline.

SEZIONE 02

Classificazione dei documenti

▲ PRIMA

Il sistema non classifica esplicitamente i documenti: ogni file viene incluso in un batch insieme ad altri 3 file, e il prompt universale di analisi (24.500 caratteri) fa da solo la classificazione implicita mentre analizza. Conseguenza: un attestato di formazione da 50 KB e un bilancio da 2,5 MB vengono trattati con la stessa logica e lo stesso prompt, e l'AI deve "indovinare" cosa sta leggendo.

Il prompt cap dinamico (`_doc_char_cap`) è basato su pattern del nome file ("visura", "dvr", "bilancio") e sui primi 1.500 caratteri del contenuto. Funziona, ma è fragile: un documento chiamato `documento_finale_completo.pdf` che contiene una visura non viene riconosciuto e riceve il cap da 12.000 chars invece di 30.000.

◆ ORA CON METODO A

Aggiungiamo uno stadio di classificazione esplicita prima dell'analisi vera. Funziona così:

1. Raccogliamo per ogni file: nome, dimensione, prime 200 parole estratte allo Stadio 1.
2. Una sola chiamata API a `gemini-2.5-flash-lite` (modello piccolo, 10× più economico del Flash standard) riceve la lista di tutti i 652 file e restituisce per ognuno un'etichetta: `VISURA` , `STATUTO` , `DVR` , `POS` , `BILANCIO` , `CCNL` , `ATTESTATO` , `CERTIFICATO_ISO` , `CONTRATTO` , `FATTURA` , `IDENTITA` , `ALTRO` .
3. La classificazione viene usata per: assegnare il cap di caratteri corretto, raggruppare i batch in modo intelligente, costruire l'intestazione del Word con un indice ordinato per macroarea.

→ PERCHÉ MIGLIORA

- **Cap caratteri corretto al 99% dei casi** invece di pattern fragili: l'AI riconosce `documento_finale_completo.pdf` come visura leggendo le prime 200 parole.
- **Batch semanticamente coerenti**: visura + statuto + atto costitutivo nello stesso batch permettono di estrarre l'identità aziendale in modo coeso, evitando contraddizioni tra batch diversi.
- **Costo trascurabile**: la chiamata di classificazione consuma circa 5.000 token su un modello che costa 0,075 € per milione di token in input. Per ogni ZIP: circa **0,0004 €**. Praticamente gratis.

SEZIONE 03

Invio del prompt universale

▲ PRIMA

Il file `webapp/prompts/universal_evidence_prompt.md` contiene 24.561 caratteri di istruzioni (le SOP, le regole di output, gli schemi YAML, le forme giuridiche italiane, ecc.). Equivale a circa **6.100 token**.

Questo prompt viene concatenato in testa a ogni singolo batch prima di inviarlo a Gemini. Sulla ZIP boss, con 88 batch, il sistema invia **$88 \times 6.100 = 537.000$ token** di istruzioni identiche, riletti dal modello da capo ogni volta.

A 0,30 € per milione di token in input (Gemini 2.5 Flash), questo è **0,16 € sprecati per ZIP** solo nel reinviare lo stesso testo. Su 30 audit/mese: 4,80 €/mese, **58 €/anno**, su istruzioni che non cambiano mai.

◆ ORA CON METODO A

Usiamo la funzionalità **Context Caching** di Gemini API (rilasciata da Google nel 2024, attiva di default da maggio 2025).

1. All'avvio del processo, il prompt universale viene caricato una volta sola nel cache di Google con `client.caches.create(...)`. Riceve un identificativo (es. `cachedContents/abc123`).
2. Ogni batch successivo invia solo i documenti specifici e fa riferimento al `cache_id`.
3. Google fattura il prompt una volta sola all'upload, e ogni rilettura costa -90% rispetto al normale (sui modelli Gemini 2.5).

→ PERCHÉ MIGLIORA

- **Risparmio token:** $88 \times 6.100 = 537.000 \rightarrow 88 \times 610$ (con sconto 90%) = 53.680. Risparmio reale: **483.320 token per ZIP**.
- **Latenza inferiore:** il modello non rilegge il prompt da zero, riusa lo stato cached. Stima: -15-20% sul tempo di risposta per batch (osservato in benchmark Google).
- **Costo annuo:** lo sconto del 90% sul prompt cached, applicato a 30 audit/mese, vale circa **45-50 € l'anno** solo su questa voce.
- **Storage del cache:** \$1,00 per milione di token/ora sul Flash. Il nostro prompt = 6.100 token = 0,006 €/ora di storage. Tenuto attivo 8 ore al giorno: 0,05 €/giorno = **1,50 €/mese**. Si ricrea automaticamente alla scadenza con un cron.

SEZIONE 04

Upload delle immagini per OCR

▲ PRIMA

Quando un PDF va in OCR, il sistema:

1. Apre il PDF con `PyMuPDF`, lo converte pagina per pagina in immagini PNG a 800 px.
2. Ogni immagine viene codificata in **base64** (formato testuale che gonfia i dati del 33%).
3. Il base64 viene allegato in linea (`inline_data`) al prompt API.

Su un PDF da 12 pagine: 12 immagini $\times$ ~150 KB/PNG $\times$ 1,33 = **~2,4 MB di base64** in un singolo payload API. Per 60 PDF in OCR: **~144 MB di traffico HTTP cumulato**. Su rete Hetzner congestionata o proxy Cloud Functions, questo causa timeout, errori 503 e ritrasmissioni.

Inoltre, se la stessa immagine va analizzata 2 volte (es. retry su rate limit 429), viene ricaricata da zero ogni volta.

◆ ORA CON METODO A

Usiamo la **Files API di Gemini** (`client.files.upload()`).

- Le immagini vengono caricate una volta sola sui server di Google. Ricevono un URI riferibile (es. `files/abc123`).
- Le chiamate API successive passano l'URI invece dei bytes. Payload ridotto da MB a poche centinaia di byte.
- Se serve un retry, l'URI è già attivo, nessuna ri-codifica.
- I file caricati vivono 48 ore su server Google, poi si auto-cancellano.
- Possibilità di caricare PDF interi fino a 2 GB (vs 100 MB del base64 inline) — utile per file da 60 MB come il `RELAZIONE_POS_REV_05.pdf` .

→ **PERCHÉ MIGLIORA**

- **Niente più crash da payload too large:** oggi se un PDF supera certi limiti del proxy, l'OCR fallisce in modo opaco. Con Files API il limite è **2 GB**.
- **Latenza upload parallelizzabile:** 60 file possono essere caricati in 5 worker async mentre la pipeline fa altro. Stima: **-30-50%** sul tempo OCR totale.
- **Riduzione errori 503:** meno bytes in transito = meno congestione su Cloud Functions proxy. Sulla baseline aprile: 7-12% di chiamate fallite con 503; atteso scendere **sotto il 2%**.
- **Contesto intero del documento:** possibilità di mandare il PDF intero invece che spezzettarlo in immagini pagina per pagina. Su file con tabelle complesse, l'AI vede il documento nel contesto giusto.

SEZIONE 05

Generazione dell'output (Streaming)

▲ PRIMA

Per ogni batch, il sistema chiama `client.models.generate_content(...)` e attende la risposta completa. Tempo medio: **25–45 secondi** per batch su Gemini Flash, 60–90 secondi se Google è in carico. Durante questo tempo, l'utente vede solo "Analisi batch 12/88..." e una progress bar che aumenta a salti.

Se la risposta è gigantesca (es. il famoso Batch 64 della pratica boss: **1.021.244 caratteri**), il client riceve la risposta in un colpo solo, la passa al parser YAML, poi al builder docx, e a quel punto il worker unicorn crasha perché lxml costruisce un albero XML troppo grande in memoria. Tutto avviene a porte chiuse: l'utente continua a vedere "Analisi batch 64/88..." finché il timeout SSE non chiude la connessione (5–10 minuti), poi schermata vuota, niente errore, niente Word.

◆ ORA CON METODO A

Usiamo la modalità **streaming** dell'API Gemini (`generate_content_stream`):

1. Il modello inizia a generare YAML token per token.
2. Ogni chunk di tokens (50–200 caratteri) viene immediatamente inviato al frontend via SSE come evento `llm.token` e accumulato in un buffer server-side.
3. Mentre il modello sta ancora scrivendo, il frontend mostra il testo che sta arrivando. L'utente vede l'audit costruirsi in tempo reale.

Aggiungiamo un **cap rigido di sicurezza**: se il buffer di un singolo batch supera 400.000 caratteri, lo tronchiamo, registriamo un evento di warning, e proseguiamo. Niente più crash silenziosi da output mostruosi.

→ PERCHÉ MIGLIORA

- **Percezione di velocità**: anche se il tempo totale fosse identico, vedere il testo che scorre cambia totalmente l'esperienza utente. Niente più "loading infinito".
- **Diagnostica precoce**: se un batch è andato male, lo vediamo entro i primi 500 caratteri della risposta, non dopo 90 secondi.
- **Cap protettivo automatico**: il caso boss (risposta da 1 MB) non potrà più crashare il sistema. La risposta viene troncata a 400.000 caratteri con un avviso.
- **Cancellazione possibile**: se l'utente capisce a metà che ha caricato la ZIP sbagliata, può chiudere il tab e il server interrompe lo stream. Oggi il batch va a finire comunque.

SEZIONE 06

Costruzione del Word

▲ PRIMA

Tutti i batch finiscono di rispondere → si concatenano in un mega-YAML → si parse → si costruisce un singolo `Document()` con `python-docx` → `doc.save()` su un file temporaneo → il file viene letto come bytes e codificato in base64 → un unico evento SSE `done` invia **5–10 MB di base64** al browser.

Tre fragilità in serie:

1. Se `doc.save()` crasha (lxml/segfault su `parsed_data` troppo grandi), perdi tutto il lavoro fatto. Il caso boss.
2. Se uno dei 88 batch non ha risposto, il YAML aggregato ha buchi e il parser può abortire l'intera generazione.
3. Il base64 da 5–10 MB in un singolo SSE message stressa il parser JSON del browser e può causare errori `Maximum call stack size exceeded` su Chrome.

◆ ORA CON METODO A

Cambiamo da costruzione monolitica a **costruzione incrementale**:

1. Appena un gruppo di batch correlati è completo (es. tutti i batch della macroarea "DOCUMENTAZIONE LEGALE"), il sistema genera un `.docx` parziale che contiene solo quella sezione. Lo salva in `temp/sections/{session_id}/01_legale.docx`.
2. Emette un evento SSE `section.ready` con un URL di download anticipato.
3. Quando arrivano tutti i pezzi, usa la libreria `docxcompose` (testata su grandi documenti) per fare il merge incrementale dei `.docx` parziali. Niente albero XML costruito a mano in memoria.
4. Il file finale viene scritto su disco e il client riceve solo l'URL di download, non i bytes inline. Niente più base64 da MB in SSE.

→ **PERCHÉ MIGLIORA**

- **Niente più crash atomici:** se la sezione "BILANCI" ha un batch problematico, perdi solo quella sezione, le altre 9 si salvano comunque.
- **Download anticipati:** se il cliente vuole già la parte legale mentre quella sicurezza sta ancora generando, può scaricarla. Riduce l'attesa percepita.
- **Stabilità su Word grandi:** documenti da 80–120 pagine (audit completo di azienda strutturata) costruiti via `docxcompose` non hanno il limite di lxml.
- **Recoverability:** se cade la rete a metà, alla riconnessione il client trova le sezioni già pronte sul server e le scarica senza ri-elaborare nulla.

SEZIONE 07

Gestione errori e crash silenziosi

▲ PRIMA

Il blocco `try / except Exception` nel handler SSE (`api_server.py:567`) cattura solo le eccezioni Python normali. Non cattura:

- `MemoryError` (memoria esaurita)
- Crash a livello C (lxml/segfault, OOM kernel)
- Morte del worker uvicorn (es. SIGKILL da OOM-killer Linux)

Quando uno di questi accade — come sulla pratica boss — il worker muore senza eseguire il blocco `except`. Il client SSE smette di ricevere eventi. Niente errore inviato al frontend. Il browser pensa che il server sia ancora vivo e l'utente vede "Analisi in corso..." per ore.

◆ ORA CON METODO A

Tre livelli di protezione attivati in cascata:

1. `except BaseException` invece di `except Exception` : cattura anche `MemoryError` , `SystemExit` , `KeyboardInterrupt` . Emette evento SSE `error` di tipo specifico prima che il processo termini.
2. **Heartbeat watchdog:** un thread separato emette ogni 5 secondi un evento heartbeat con PID, memoria RAM usata, profondità della coda. Se il frontend non riceve heartbeat per 15 secondi, mostra automaticamente un avviso e tenta una riconnessione su endpoint `/api/report/resume/{session_id}` .
3. **Persistenza dello stato su disco:** ogni evento SSE viene anche scritto in `temp/progress/{session_id}.json` . Se il worker muore, l'endpoint `resume` legge il file di stato e dice al client cosa è stato completato. L'utente non perde la visibilità.

In aggiunta, prima ancora di arrivare al crash: cap di sicurezza sulla risposta API (se supera 400.000 caratteri, viene troncata); cap sul YAML aggregato (se supera 6 MB, blocchiamo e segnaliamo); **subprocess isolation** via `multiprocessing.Process` con Pipe di comunicazione.

→ **PERCHÉ MIGLIORA**

- **Zero crash silenziosi:** ogni errore, anche quelli a livello sistema, produce un evento utente-visibile entro 15 secondi.
- **Cause specifiche per ogni errore:** invece di generico "errore generazione", l'utente vede messaggi come "risposta AI troppo grande sul batch 64 — troncata", "memoria insufficiente — riduci la dimensione della ZIP". Riduce le ticket di supporto al **90%**.
- **Recovery senza perdere lavoro:** anche dopo un crash, il sistema sa cosa è stato fatto e può ripartire da dove si è fermato.

SEZIONE 08

Telemetria real-time per l'utente

▲ PRIMA

Il frontend riceve due tipi di evento SSE: `{pct: 45, msg: "Analisi batch 12/88..."}` e l'evento finale `done`. La progress bar è quasi unica fonte di feedback. L'utente non sa: quale file specifico viene processato, se un file ha fallito la lettura, se un file è stato degradato (OCR fast mode da 3 pagine invece di 12), né se il sistema ha rilevato anomalie nella ZIP.

◆ ORA CON METODO A

Tipizzazione degli eventi SSE in **12 categorie distinte**. Esempio di cosa vedrebbe l'utente in tempo reale:

```
14:32:01 Inizio elaborazione di "Documenti_Audit_2026.zip" (652 file, 381 MB)
14:32:03 Estrazione completata: 652 file in 4 cartelle
14:32:03 Avviso: 1 file > 50 MB ("RELAZIONE_POS.pdf"), processerò max 12 pagine
14:32:08 Classificazione: 64 visure/statuti, 218 attestati, 47 contratti, 23 DVR/POS
14:32:09 Lettura PDF digitali: 487/652 file (75%) — testo nativo
14:33:21 Lettura completata. Avvio OCR per 165 file scansionati...
14:36:45 OCR completati: 162/165. Falliti: "scan_corrotto.pdf" (file danneggiato)
14:36:46 Avvio analisi AI strutturata. Prompt cached: -90% costo
14:36:51 Sezione [01 LEGALE] in elaborazione (8 documenti)
14:37:33 Sezione [01 LEGALE] completata ✓ — Scarica anteprima
14:37:34 Sezione [02 CONTRIBUTIVA] in elaborazione (11 documenti)
...
14:51:22 Sezione [10 ALTRO] completata ✓
14:51:24 Generazione Word finale...
14:51:33 Audit_DEMOCLIENT_20260430_145133.docx pronto — Scarica
```

→ PERCHÉ MIGLIORA

- **Trasparenza totale:** il cliente sa cosa il sistema sta facendo in ogni momento. Niente più "ma è ancora vivo?"
- **Reazione precoce:** se vede "1 file >50 MB", sa che quella elaborazione ci metterà di più, e non si allarma a 5 minuti dall'inizio.
- **Download incrementale:** ogni sezione completata è scaricabile. Se ne servono solo 2 su 10, può fermarsi prima.
- **Diagnostica self-service:** se "scan_corrotto.pdf" è fallito, lo sa subito e può riprovare con quel singolo file riparato senza rifare l'intera ZIP.

SEZIONE 09

Sintesi numerica — Prima / Ora

Numeri calcolati sulla pratica boss del 29 aprile (652 file, 381 MB) e sul carico medio (200 file, 150 MB) di un audit standard.

AUDIT STANDARD — 200 FILE

Metrica	Prima	Con Metodo A	Variazione
Tempo totale	12–15 min	3–4 min	–72%
Token Gemini consumati	~1,8 milioni	~250.000	–86%
Costo Gemini per audit	~1,40 €	~0,20 €	–86%
Chiamate OCR Vision	~80–100	~20–30	–70%
Tempo primo feedback all'utente	8–12 sec	< 1 sec	–92%
Tasso silent-failure	~2–3%	< 0,1%	Eliminato

AUDIT GRANDE — CASO BOSS (652 FILE)

Metrica	Prima	Con Metodo A	Variazione
Tempo totale	25–40 min (o crash)	6–9 min	–75%
Token Gemini consumati	~3,5–4,5 milioni	~600.000	–85%
Costo Gemini per audit	~3,00 €	~0,45 €	–85%
Chiamate OCR Vision	~318	~60–80	–78%
Probabilità completamento riuscito	~70%	> 99%	+29 punti

PROIEZIONE ANNUALE — 30 AUDIT/MESE, DIMENSIONI MISTE

Voce	Prima	Con Metodo A	Risparmio annuo
Costo Gemini API	~720 €	~108 €	~612 €
Tempo operatore (attese audit)	360 ore/anno	90 ore/anno	270 ore
Crash su pratiche grandi	~12 incidenti/anno	~0	–12 incidenti

SEZIONE 10

Riepilogo per il cliente

Sintesi in 5 punti operativi — tutto il resto del documento fornisce la base tecnica di supporto.

01 – Nessuna perdita di precisione

Il sistema continua a leggere ogni documento del cliente con la stessa profondità di oggi. Non si perde precisione: anzi, sui PDF nativi miglioriamo perché non passano più da OCR ridondante.

02 – Tempo di audit ridotto del 70-75%

Da 12-15 minuti a 3-4 minuti per pratiche normali, da 25-40 minuti a 6-9 minuti per pratiche grandi.

03 – Costo per audit in token Gemini -85%

Grazie a 4 ottimizzazioni cumulate: lettura testo nativo dove possibile, classificazione preventiva con modello piccolo, caching del prompt grande, upload immagini condiviso.

04 – Zero crash silenziosi

I crash silenziosi non sono più possibili: ogni errore è intercettato, comunicato all'utente in tempo reale e tracciato. Se qualcosa va storto, si sa cosa, dove e perché.

05 – Il sistema lavora in trasparenza totale

L'utente vede il sistema lavorare in tempo reale: ogni file processato genera un evento, ogni sezione completata è scaricabile in anticipo. Niente più "loading infinito".

🕒 **Tempi di realizzazione:** 1-2 settimane di sviluppo + 2 giorni di test su casi storici. Lavoreremo in un branch separato del codice, attivabile con un interruttore: se qualcosa non va, si torna al sistema attuale in **30 secondi** senza perdita di dati.

✓ **Compatibilità con il lavoro fatto finora:** le SOP, i prompt esistenti, la struttura del Word di output, i template per le checklist — tutto resta invariato. Il Metodo A migliora *il come* il sistema lavora, non *il cosa* produce.

